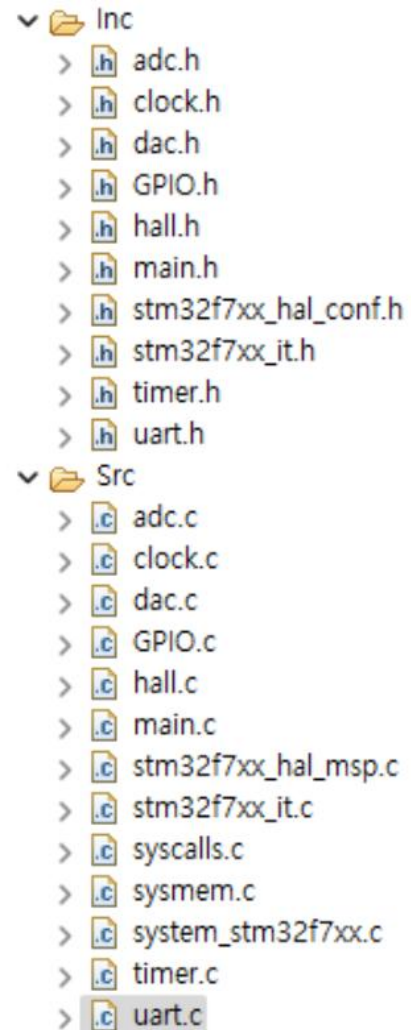


■ 헤더 분리를 통한 프로젝트 관리



■ BLDC 모터 구동

- BLDC 모터 구동을 위해선 홀 센서로 회전자의 위치를 파악하여 스위칭 소자를 On/Off 해야함
- Hall a, Hall b, Hall c는 1 또는 0의 값을 가지며, 000,111을 제외한 총 6개의 경우의 수를 가진다.
- MCU로 입력되는 홀 센서의 값에 따라서 스위치를 On/Off 함
- 모든 모터의 홀 센서 신호 출력 순서가 같지 않음

U상	0	1	1	0	-1	-1
V상	-1	-1	0	1	1	0
W상	+1	0	-1	-1	0	1
Ha	0	0	0	1	1	1
Hb	1	1	0	0	0	1
Hc	0	1	1	1	0	0
Sector	1(0~60°)	2(60~120°)	3(120~180°)	4(180~240°)	5(240~300°)	6(300~360°)
Hall Sum	2	3	1	5	4	6

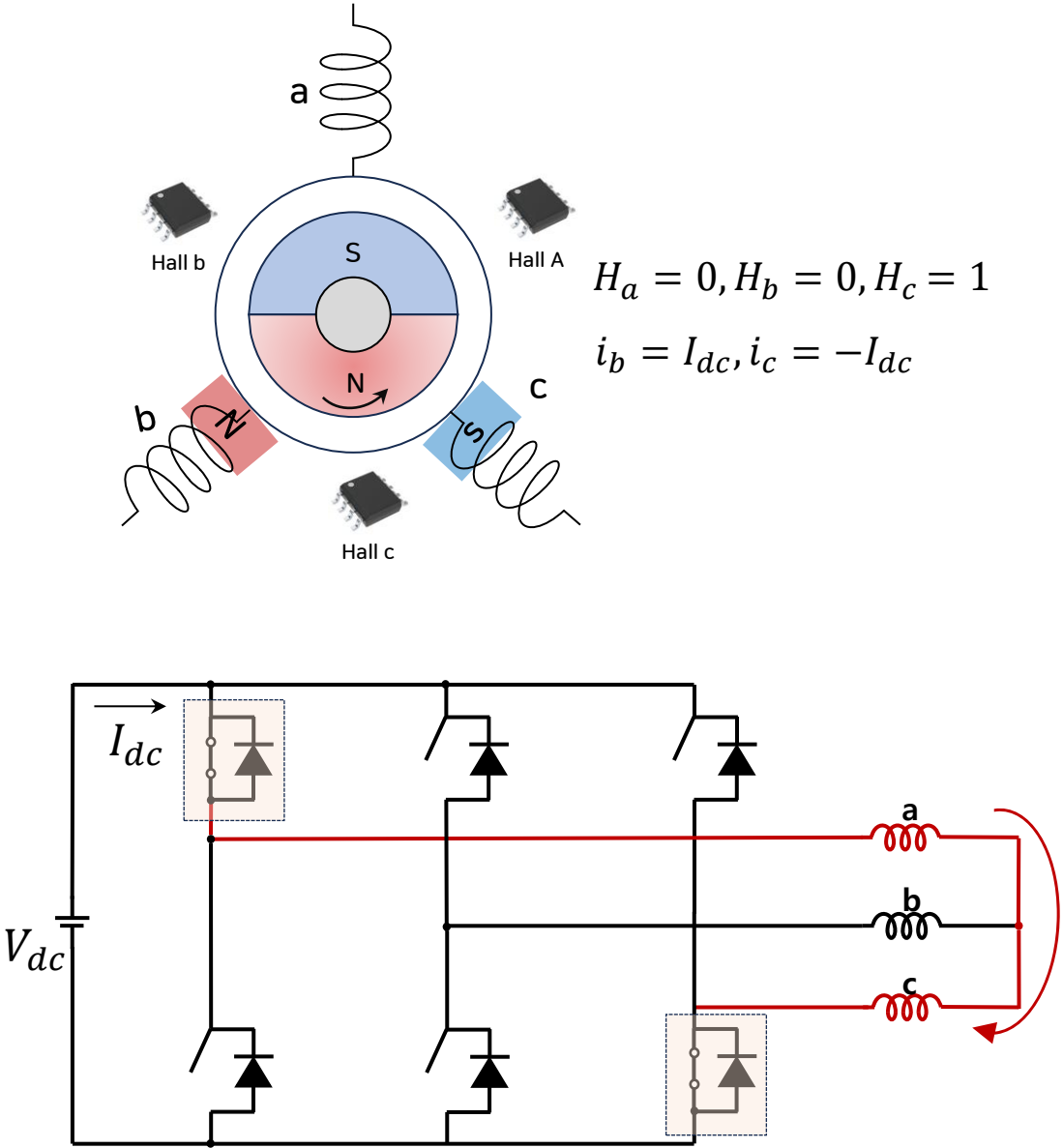
Hall sum = $H_a * 2^2 + H_b * 2^1 + H_c * 2^0$

BLDC 모터 구동

아래는 $H_a=0, H_b=0, H_c=1$ 이 되었을 때의 스위칭 신호 인가 예시

U상	0	1	1	0	-1	-1
V상	-1	-1	0	1	1	0
W상	+1	0	-1	-1	0	1
Ha	0	0	0	1	1	1
Hb	1	1	0	0	0	1
Hc	0	1	1	1	0	0
Sector	1(0~60°)	2(60~120°)	3(120~180°)	4(180~240°)	5(240~300°)	6(300~360°)
Hall Sum	2	3	1	5	4	6

$Hall\ sum = H_a * 2^2 + H_b * 2^1 + H_c * 2^0$

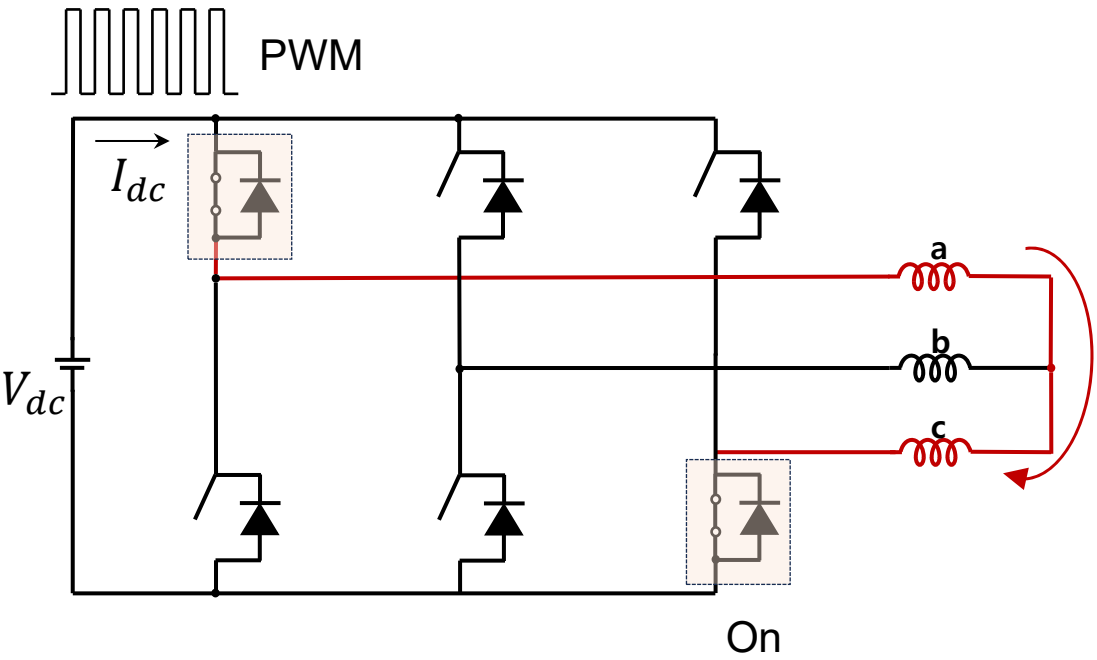


BLDC 모터 구동

- 홀 센서 변화 → 010, 011, 001, 101, 100, 110으로 변화
- MCU로 홀 센서 변화를 감지하기 위해 Rising/Falling Edge를 인터럽트로 입력 받은 뒤 미리 정해진 스위칭 패턴 On/Off
- 유니폴라 PWM 제어를 사용할 것이고, 상단 스위치를 PWM 신호로 주고, 하단 스위치는 On 함 → 상단 스위치 유니폴라 PWM 제어

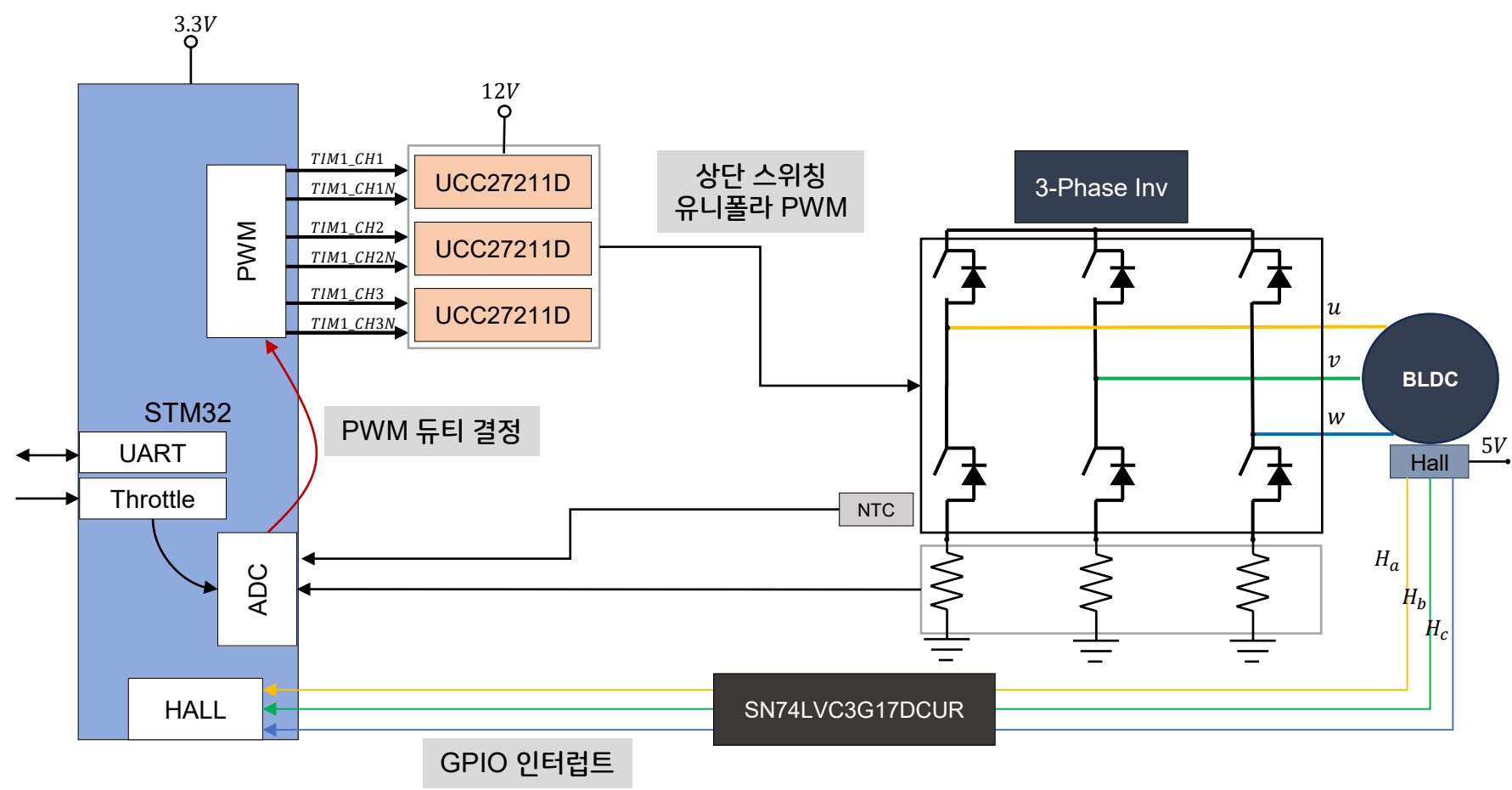
U상	0	1	1	0	-1	-1
V상	-1	-1	0	1	1	0
W상	+1	0	-1	-1	0	1
Ha	0	0	0	1	1	1
Hb	1	1	0	0	0	1
Hc	0	1	1	1	0	0
Sector	1(0~60°)	2(60~120°)	3(120~180°)	4(180~240°)	5(240~300°)	6(300~360°)
Hall Sum	2	3	1	5	4	6

$$\text{Hall sum} = H_a * 2^2 + H_b * 2^1 + H_c * 2^0$$



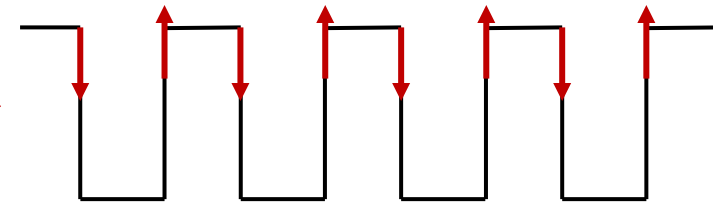
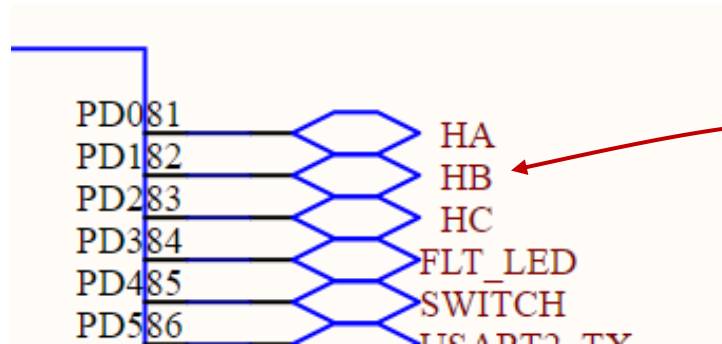
BLDC 모터 구동

- 홀 센서 GPIO로 입력 받아 Rising/Falling Edge를 검출하여 인터럽트 발생
- 읽어드린 홀 센서를 확인하고 미리 정해진 스위칭 패턴 인가
- 이때 아래 스위치는 On, 상단 스위치는 PWM 인가 → PWM Duty는 스로틀 신호에 비례하여 증가



■ BLDC 홀 센서 입력

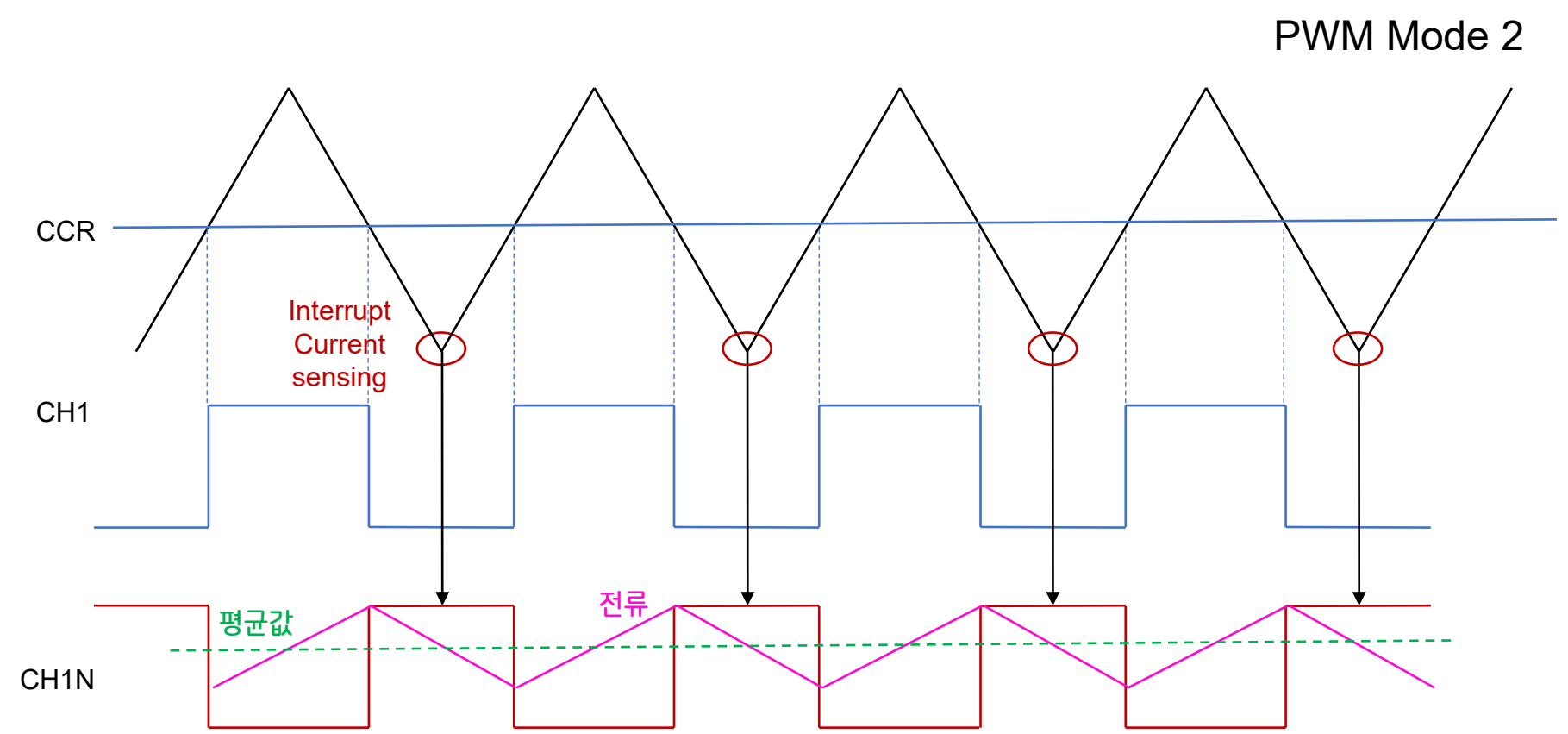
- PD0,PD1,PD2 레지스터 설정 → Input, 인터럽트 설정



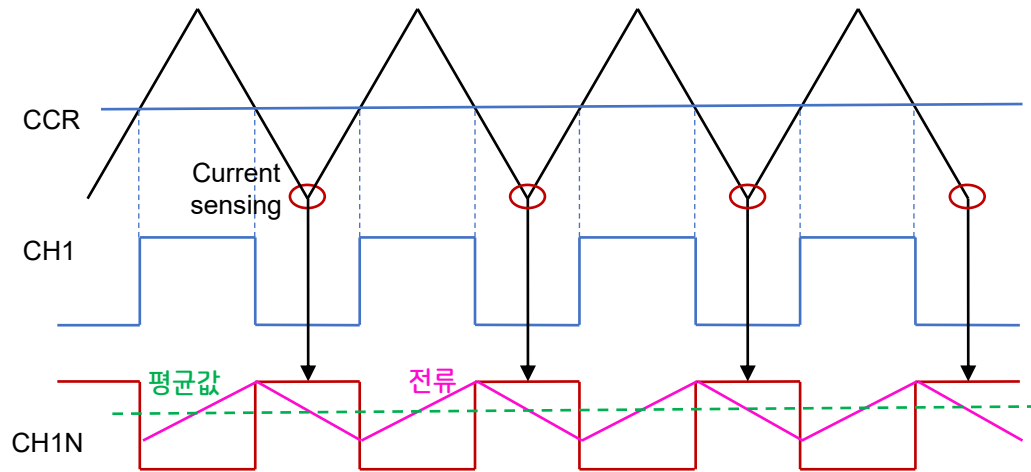
홀 센서 신호

Center Align PWM 설정

- 센트 저항으로 전류를 정확하게 읽기 위해서 Center Align PWM 사용
- 즉 카운터의 언더플로우 지점에서 인터럽트를 발생시키고, ADC를 통해 전류를 센싱 해야함
- 또한 아래 스위치가 On 되었을 때 ADC 센싱을 해야 센트 저항에 흐르는 전류를 정확히 측정할 수 있음



Center Align PWM 설정



- 카운터 동작 시작 전에 TIMx_RCR 레지스터 값을 설정하면 오버플로우, 카운터 동작이 시작된 후 TIMx_RC 레지스터 값을 설정하면 언더플로우에서 업데이트 이벤트 발생

```
#define CNT_MAX 5400 //5400
```

```
void Initialize_PWM(void) /* initialize TIM1 for PWM */
```

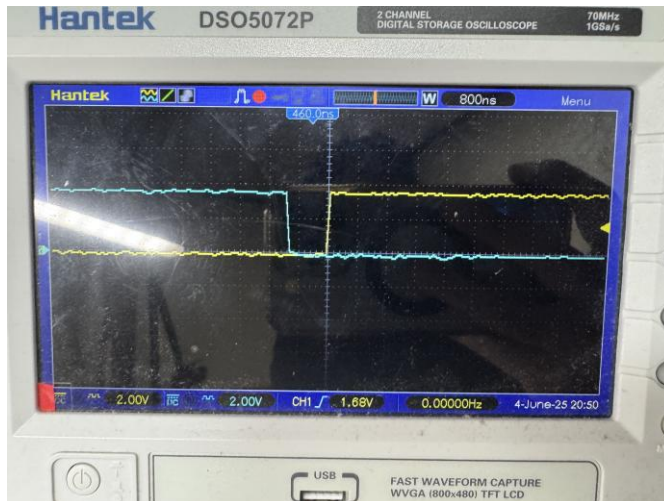
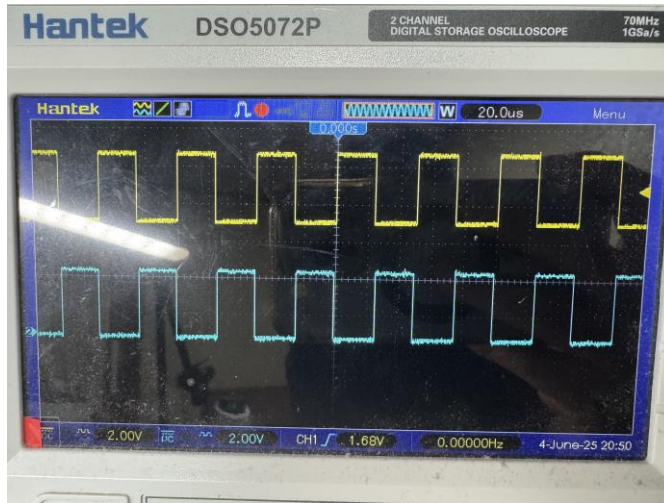
```
{
    GPIOE->MODER &= 0x0000FFFF; // use      TIM1_CH3/3N/2/2N/1/1N
    GPIOE->MODER |= 0x9AAA0000;
    GPIOE->AFR[1] |= 0x10111111; // PE13~PE8 = TIM1_CH3/3N/2/2N/1/1N
    GPIOE->OSPEEDR |= 0x0FFF0000; // PWM signal = 180MHz
    RCC->APB2ENR |= 0x00000001; // enable TIM1 clock
    //216MHz
```

```
    TIM1->PSC = 0x0000; // 216MHz/(0+1) = 216MHz
    TIM1->ARR = CNT_MAX; // 216MHz/CNT_MAX/2 = 20kHz
    TIM1->CCMR1 = 0x00007878; // TIM1_CH2/2N/1/1N = PWM mode 2
    TIM1->CCMR2 = 0x00000078; // TIM1_CH3/3N = PWM mode 2
    TIM1->CCER = 0x00000555; // TIM1_CH3/3N/2/2N/1/1N = active high
    TIM1->BDTR = 0x00001C00; // BKE = OSSR = OSSI = 1
    TIM1->BDTR |= TIM_BDTR_MOE | DEADTIME_1us;
    TIM1->DIER = 0x0001; // enable update interrupt
    TIM1->CR1 = 0x0065; // center-aligned mode 3, URS = CEN = 1
```

```
NVIC_EnableIRQ(TIM1_UP_TIM10_IRQn);
```

```
while(!(TIM1->CR1 & 0x0010)); // TIM1 underflow event ?
TIM1->RCR = 0x0001; // 50 us period update(RCR = 1)
```

■ Center Align PWM 설정



```
#define DEADTIME_1us ((uint32_t)180)
```

```
void Initialize_PWM(void) /* initialize TIM1 for PWM */
{
    GPIOE->MODER &= 0x0000FFFF; // use
    TIM1_CH3/3N/2/2N/1/1N
    GPIOE->MODER |= 0x9AAA0000;
    GPIOE->AFR[1] |= 0x10111111; // PE13~PE8 =
    TIM1_CH3/3N/2/2N/1/1N
    GPIOE->OSPEEDR |= 0x0FFF0000; // PWM signal = 180MHz
    RCC->APB2ENR |= 0x00000001; // enable TIM1 clock
    //216MHz
}
```

```
TIM1->PSC = 0x0000; // 216MHz/(0+1) = 216MHz
TIM1->ARR = CNT_MAX; // 216MHz/CNT_MAX/2 = 20kHz
TIM1->CCMR1 = 0x00007878; // TIM1_CH2/2N/1/1N = PWM
mode 2
TIM1->CCMR2 = 0x00000078; // TIM1_CH3/3N = PWM mode 2
TIM1->CCER = 0x00000555; // TIM1_CH3/3N/2/2N/1/1N =
active high
TIM1->BDTR = 0x00001C00; // BKE = OSSR = OSSI = 1
TIM1->BDTR |= TIM_BDTR_MOE | DEADTIME_1us;
TIM1->DIER = 0x0001; // enable update interrupt
TIM1->CR1 = 0x0065; // center-aligned mode 3, URS = CEN
= 1
}
```

Center Align PWM 설정

Turn-on delay time ^c	$t_{d(on)}$	$V_{DD} = 30\text{ V}, R_L = 3\ \Omega$ $I_D \cong 10\text{ A}, V_{GEN} = 10\text{ V}, R_g = 1\ \Omega$	-	24	30	ns
Rise time ^c	t_r		-	8	10	
Turn-off delay time ^c	$t_{d(off)}$		-	47	58	
Fall time ^c	t_f		-	15	19	

■ SysTick

- Cortex-M 코어에 내장된 24비트 다운 카운터 타이머
- 지정된 간격(1ms)마다 언더플로우 발생
- 운영체제나 Task 구현 시 사용

```
void SysTick_Init(void)
{
    // SYSCLK가 216MHz일 때, 1ms마다 인터럽트가 발생하도록 설정:
    // Reload = (216,000,000 / 1000) - 1 = 215999
    SysTick->LOAD = (216000000 / 1000) - 1;
    SysTick->VAL = 0; // 현재 카운터 값 초기화
    // SysTick 제어: 프로세서 클록(216MHz) 사용, 인터럽트 활성화, 카운터 시작
    SysTick->CTRL = SysTick_CTRL_CLKSOURCE_Msk |
        SysTick_CTRL_TICKINT_Msk |
        SysTick_CTRL_ENABLE_Msk;
}
```

```
void SysTick_Handler(void)
{
    msTicks++;
    Scheduler();
}
```

```
void Scheduler(void)
{
    // 1ms 태스크는 매번 실행
    Task_1ms();

    // 10ms 주기 태스크: 1ms 카운터가 10의 배수이면 실행
    if ((msTicks % 10) == 0)
    {
        Task_10msFlg=1;
    }

    // 100ms 주기 태스크: 1ms 카운터가 100의 배수이면 실행
    if ((msTicks % 100) == 0)
    {
        Task_100msFlg=1;
    }

    // 500ms 주기 태스크: 1ms 카운터가 1000의 배수이면 실행
    if ((msTicks % 500) == 0)
    {
        Task_500msFlg=1;
    }

    // 1초 주기 태스크: 1ms 카운터가 1000의 배수이면 실행
    if ((msTicks % 1000) == 0)
    {
        Task_1sFlg=1;
    }
}
```

▪ SysTick

```
void Scheduler(void)
{
    // 1ms 태스크는 매번 실행
    Task_1ms();

    // 10ms 주기 태스크: 1ms 카운터가 10의 배수이면 실행
    if ((msTicks % 10) == 0)
    {
        Task_10msFlg=1;
    }

    // 100ms 주기 태스크: 1ms 카운터가 100의 배수이면 실행
    if ((msTicks % 100) == 0)
    {
        Task_100msFlg=1;
    }
    // 500ms 주기 태스크: 1ms 카운터가 1000의 배수이면 실행
    if ((msTicks % 500) == 0)
    {
        Task_500msFlg=1;
    }
    // 1초 주기 태스크: 1ms 카운터가 1000의 배수이면 실행
    if ((msTicks % 1000) == 0)
    {
        Task_1sFlg=1;
    }
}
```

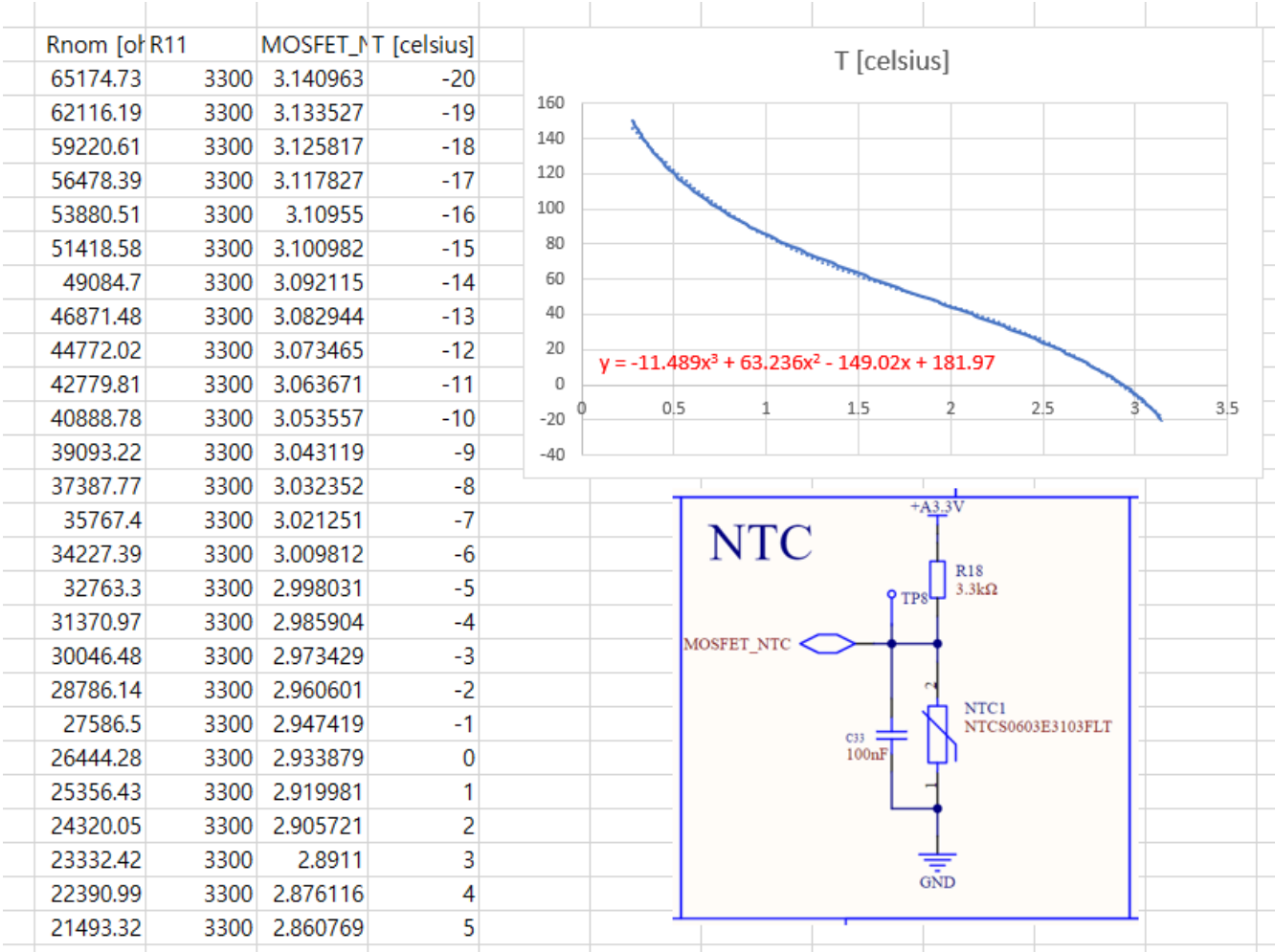
```
int main()
{
    while(1)
    {

        if(Task_1sFlg==1)
        {
            Task_1sec();
        }
        else if(Task_10msFlg==1)
        {
            Task_10ms();
        }
        else if(Task_100msFlg==1)
        {
            Task_100ms();
        }
        else if(Task_500msFlg==1)
        {
            Task_500ms();
        }
        else
        {
        }
    }
}
```

■ adc 설정

- Abc상 전류, 배터리 전압(Vdc), 쓰로틀 신호, NTC를 adc로 센싱
- 3상 전류, 쓰로틀 신호는 타이머/카운터 인터럽트에서 동시에 센싱
- 배터리전압(Vdc), NTC 온도는 10ms 마다 센싱

- NTC 온도 센싱
 - NTC 데이터시트의 정보를 이용하여 3차 다항식 추출



■ 전류 센서 오프셋 보상

- 센서의 오프셋을 보상하기 위해 처음 전원이 인가되고 난 후 10번의 평균치를 추출하여 추후 ADC 전류 센싱 과정에서 보상

```
for(int i = 0; i < 10; i++)
{
    // PA0(ias) 읽기 - ADC1
    ADC1->SQR3 = 0x00000000; // SQ1=0 (채널 0)
    ADC1->CR2 |= 0x40000000;
    while(!(ADC1->SR & 0x00000002));
    Ias_Offset += ADC1->DR;
```

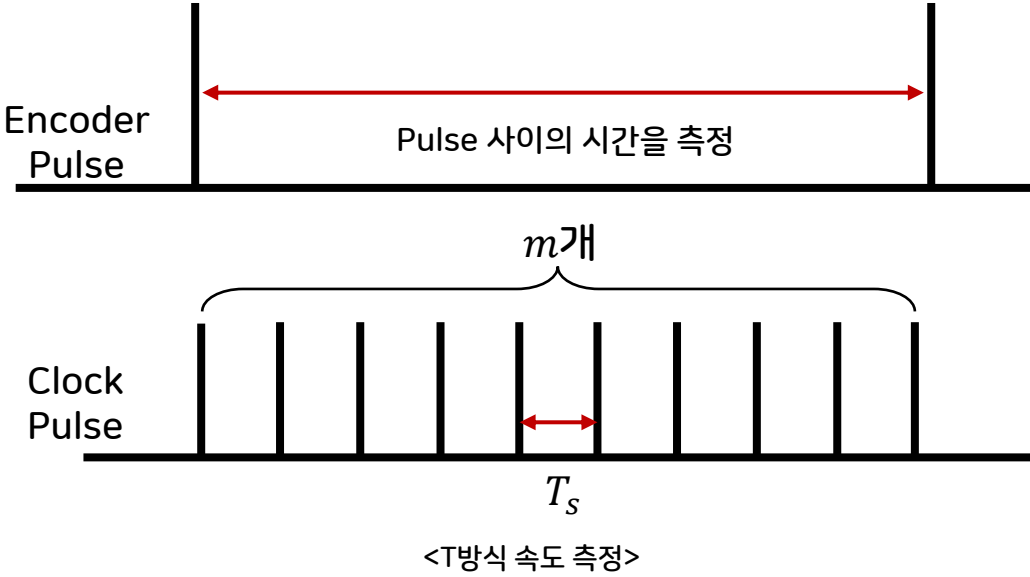
```
    // PA1(ibs) 읽기 - ADC2
    ADC2->SQR3 = 0x00000001; // SQ1=1 (채널 1)
    ADC2->CR2 |= 0x40000000;
    while(!(ADC2->SR & 0x00000002));
    Ibs_Offset += ADC2->DR;
```

```
    // PA2(ics) 읽기 - ADC3
    ADC3->SQR3 = 0x00000002; // SQ1=2 (채널 2)
    ADC3->CR2 |= 0x40000000;
    while(!(ADC3->SR & 0x00000002));
    Ics_Offset += ADC3->DR;
}
```

```
// 평균 오프셋 계산
Ias_Offset = (Ias_Offset/10) - 2048;
Ibs_Offset = (Ibs_Offset/10) - 2048;
Ics_Offset = (Ics_Offset/10) - 2048;
```

T방식의 속도 계산

- T방식의 속도 계산을 위해 타이머2를 이용하여 54MHz의 카운터 생성
- 엔코더를 사용하지 않고, 홀센서의 신호 변화를 감지하여 펄스 사이의 간격을 측정하여 속도계산 할 수 있음
- 펄스 사이의 클럭이 많이 존재할수록 분해능이 높아짐



$$\omega_m = \frac{2\pi}{m \times T_s} \frac{1}{P} [rad/s]$$
$$N = \frac{60}{m \times T_s} \frac{1}{P} [r/min]$$

P : PPR, Pulse Per Revolution
 m : 펄스 개수
 T_s : 샘플링 주기

```
void Initialize_TIM2(void)
{
    // TIM2 클럭 활성화, 54Mhz
    RCC->APB1ENR |= RCC_APB1ENR_TIM2EN;

    // TIM2 설정: 프리스케일러 1, 업카운터 모드, Auto-reload
    54000
    TIM2->PSC = 1; // 프리스케일러 = 1 → 타이머 클럭 = 108MHz /
    (1 + 1) = 54MHz
    TIM2->ARR = 53999999; // 오토리로드 = 54000 → 주기 =
    (54000 + 1) / 54MHz ≈ 1초
    TIM2->CNT = 0; // 초기 카운터 값

    // 타이머 시작
    TIM2->CR1 |= TIM_CR1_CEN;
}
```

■ T방식의 속도 계산

- 타이머2의 카운터 값을 이용하여 홀 센서 신호가 바뀌는 구간의 시간을 측정하여 모터 RPM 계산
- 홀센서 신호 인터럽트 함수에서 SpeedCal 함수 실행

```
void EXTI0_IRQHandler(void) //PORTD0 HA
{
    if (EXTI->PR & EXTI_PR_PR0)
    {
        EXTI->PR |= EXTI_PR_PR0; // 인터럽트 플래그 클리어
        Update_Hall_Sequence();
        SpeedCal();
    }
}

void EXTI1_IRQHandler(void) //PORTD1 HB
{
    if (EXTI->PR & EXTI_PR_PR1)
    {
        EXTI->PR |= EXTI_PR_PR1; // 인터럽트 플래그 클리어
        Update_Hall_Sequence();
        SpeedCal();
    }
}

void EXTI2_IRQHandler(void) //PORTD2 HC
{
    if (EXTI->PR & EXTI_PR_PR2)
    {
        EXTI->PR |= EXTI_PR_PR2; // 인터럽트 플래그 클리어
        Update_Hall_Sequence();
        SpeedCal();
    }
}
```

```
void SpeedCal(void)
{
    // 현재 타이머 값 읽기
    uint32_t current_time = TIM2->CNT;

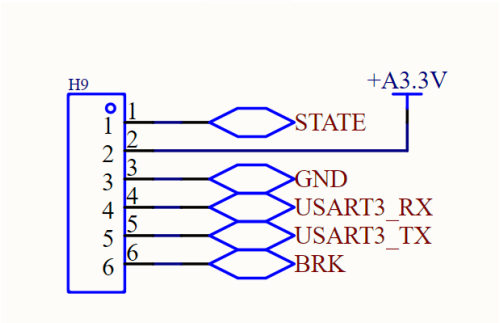
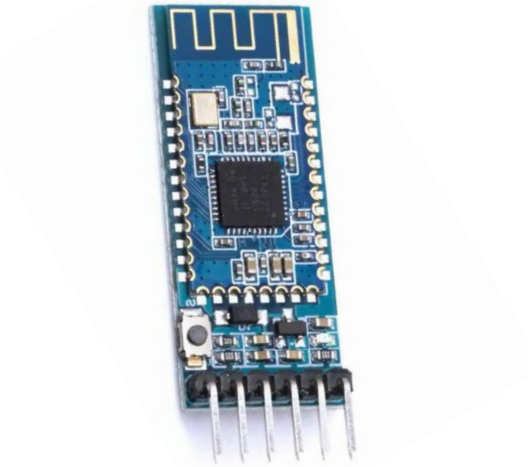
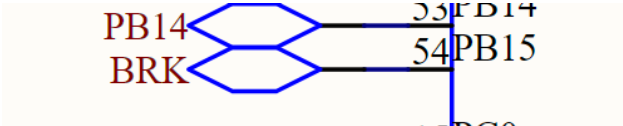
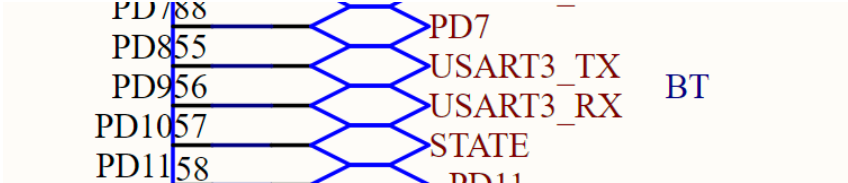
    // 타이머 오버플로우 처리
    if (current_time >= last_hall_time)
    {
        delta_time = current_time - last_hall_time;
    }
    else
    {
        // 오버플로우 발생: 타이머가 ARR 값에 도달하여 리셋됨
        delta_time = (TIM2->ARR - last_hall_time) + current_time + 1;
    }

    last_hall_time = current_time;

    // delta_time이 0이 아닐 때만 RPM 계산
    // 가정: 한 회전당 6개의 홀 센서 이벤트 발생 (6 edges per revolution)
    // Timer 주파수: 54MHz
    // RPM 계산 공식: RPM = (60 * Clock_Frequency) / (Edges_per_Revolution * delta_time)
    if(delta_time <= 50000)
    {
        //너무 빠른 신호는 무시
    }
    else
    {
        calculated_rpm = (60.0f * 54000000.0f) / ((Edges_per_Revolution * (float)delta_time));
    }
}
```

■ 블루투스 모듈(AT-09)

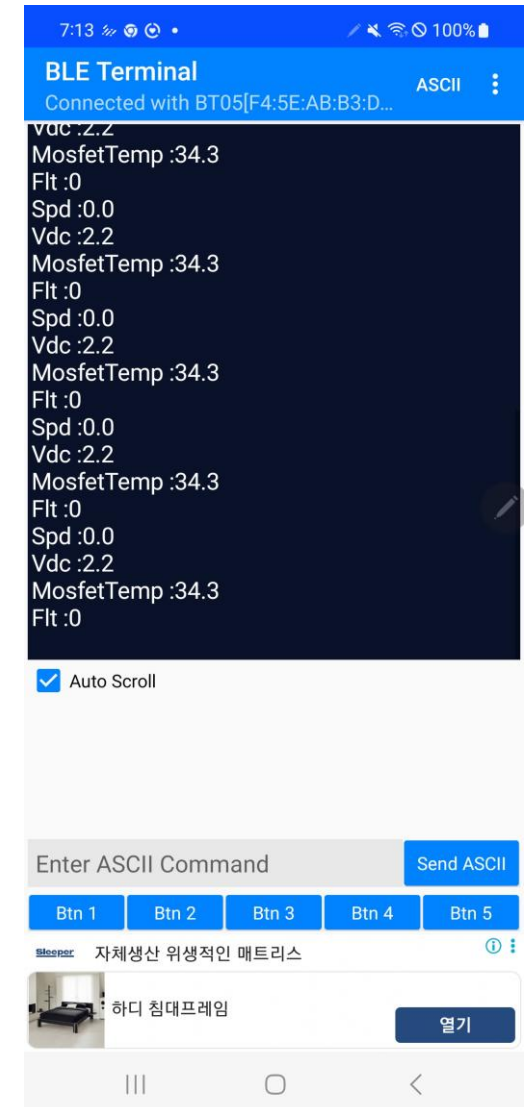
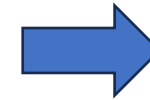
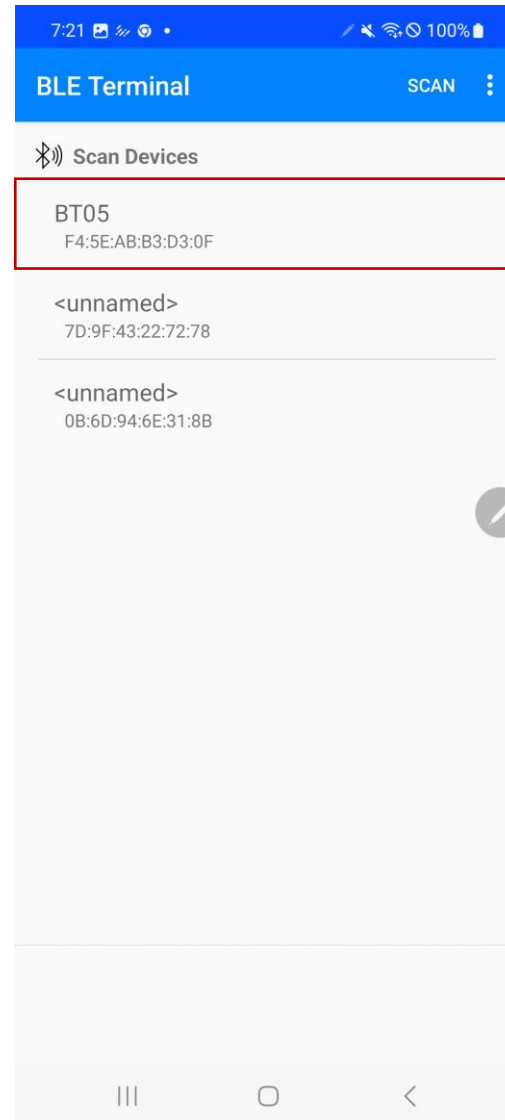
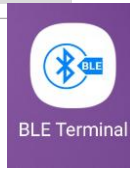
- 블루투스 모듈은 스마트폰과 데이터를 무선으로 주고 받고, 블루투스 모듈과 MCU는 UART로 통신을 수행
- AT-09 모듈은 안드로이드, IOS 모두 지원
- BRK : 블루투스 통신 중단 가능
- STATE : 블루투스 상태 확인



스마트폰

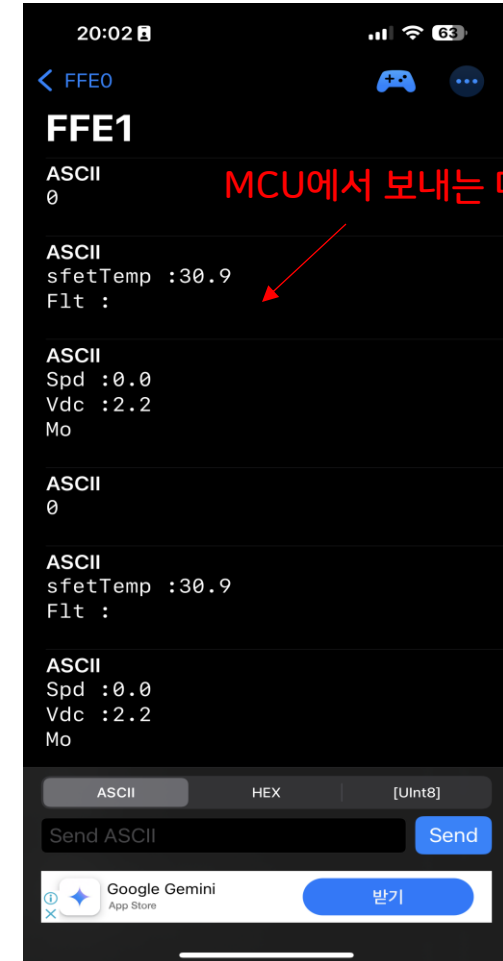
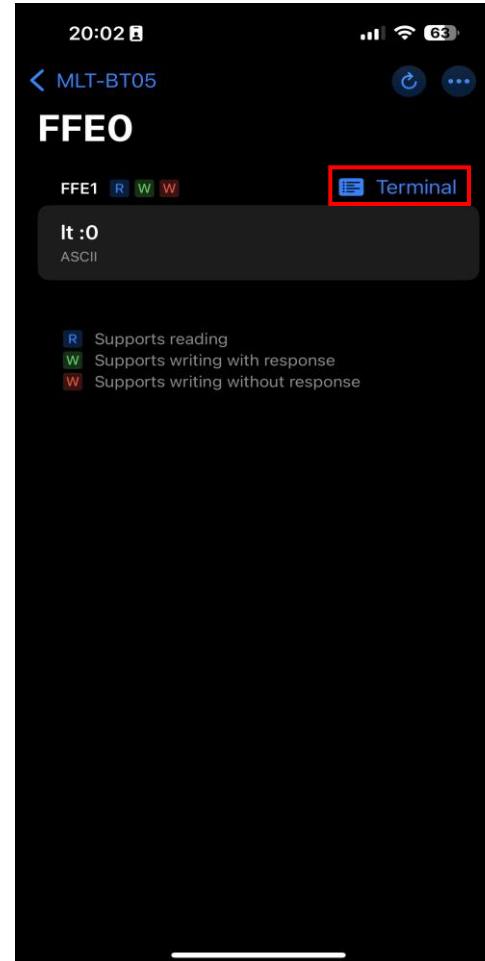
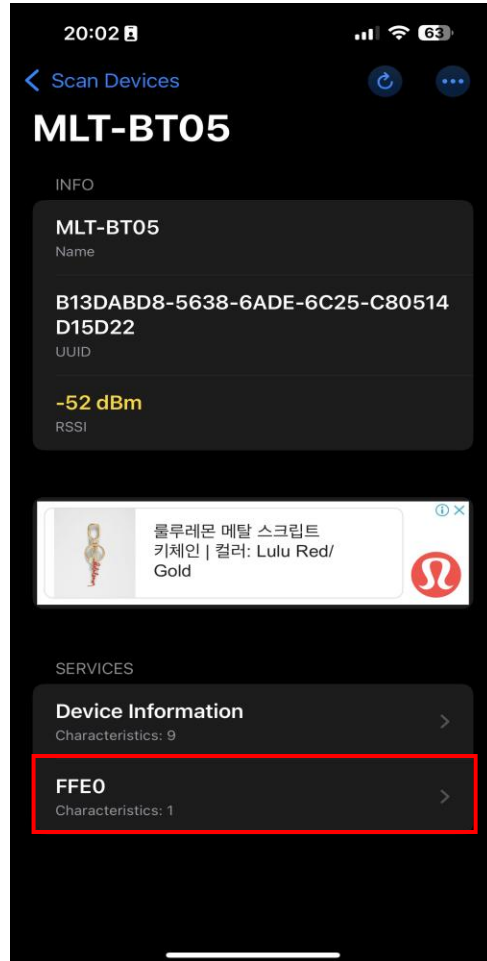
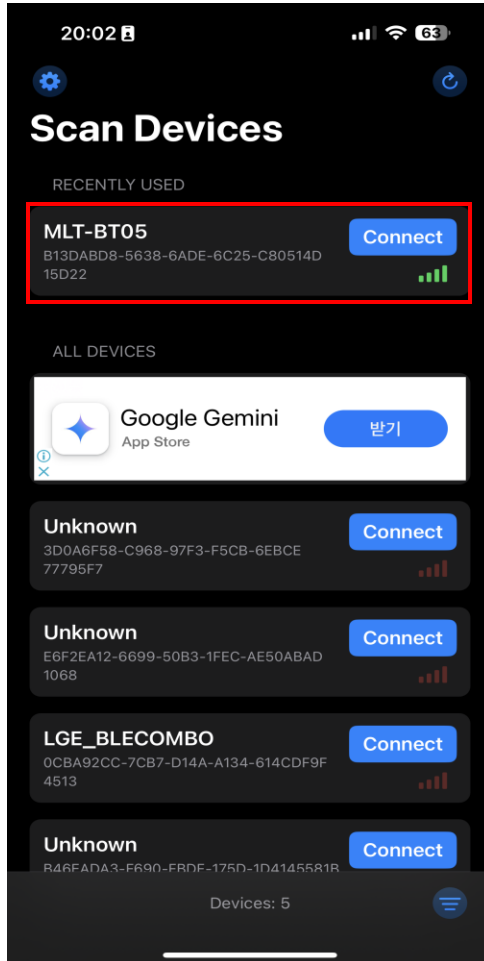
■ 블루투스 모듈 어플 (안드로이드)

- BLE Terminal 어플 다운로드
- 스마트폰 블루투스 켜고 연결하면 통신 모니터링 가능
- 초기 비밀번호 : 000000



■ 블루투스 모듈 어플 (IOS)

- BluetoothLE 어플 다운로드
- 스마트폰 블루투스 켜고 아래 순서대로 연결하면 통신 모니터링 가능
- 초기 비밀번호 : 000000



MCU에서 보내는 데이터 모니터링

■ 블루투스 모듈(AT-09)

- 통신 속도는 9600bps로 설정
- 500ms 마다 MCU의 데이터를 송신해주도록 코드 구현

```
void AT09_Init(void) // BT
{
    // 1) USART3 클럭 활성화

    RCC->APB1ENR |= RCC_APB1ENR_USART3EN;

    // 2) GPIOD 클럭 활성화 (PD8, PD9)
    RCC->AHB1ENR |= RCC_AHB1ENR_GPIODEN;

    // 3) PD8(TX), PD9(RX) → Alternate Function AF7
    GPIOD->MODER &= ~((3UL << (8 * 2)) | (3UL << (9 * 2)));
    GPIOD->MODER |= ((2UL << (8 * 2)) | (2UL << (9 * 2)));

    GPIOD->AFR[1] &= ~(0xFUL << ((8 - 8) * 4)) | (0xFUL << ((9 - 8) * 4));
    GPIOD->AFR[1] |= ((7UL << ((8 - 8) * 4)) | (7UL << ((9 - 8) * 4)));

    // 4) USART3 레지스터 초기화
    USART3->CR1 = 0;
    USART3->CR2 = 0;
    USART3->CR3 = 0;

    // 5) Baud Rate 9600bps @ APB1=54MHz → BRR = 5625
    USART3->BRR = 5625;

    // 6) TE=1, RE=1, UE=1
    USART3->CR1 |= (USART_CR1_TE | USART_CR1_RE);
    USART3->CR1 |= USART_CR1_UE;
}
```

```
void Task_500ms(void)
{
    // 블루투스 송신
    USART3_SendString("Spd :");
    UART3_SendFloat_Simple(speed_kmh,1);
    USART3_SendString("\n");

    USART3_SendString("Vdc :");
    UART3_SendFloat_Simple(Vdc,1);
    USART3_SendString("\n");

    USART3_SendString("MosfetTemp :");
    UART3_SendFloat_Simple(MosfetTemp,1);
    USART3_SendString("\n");

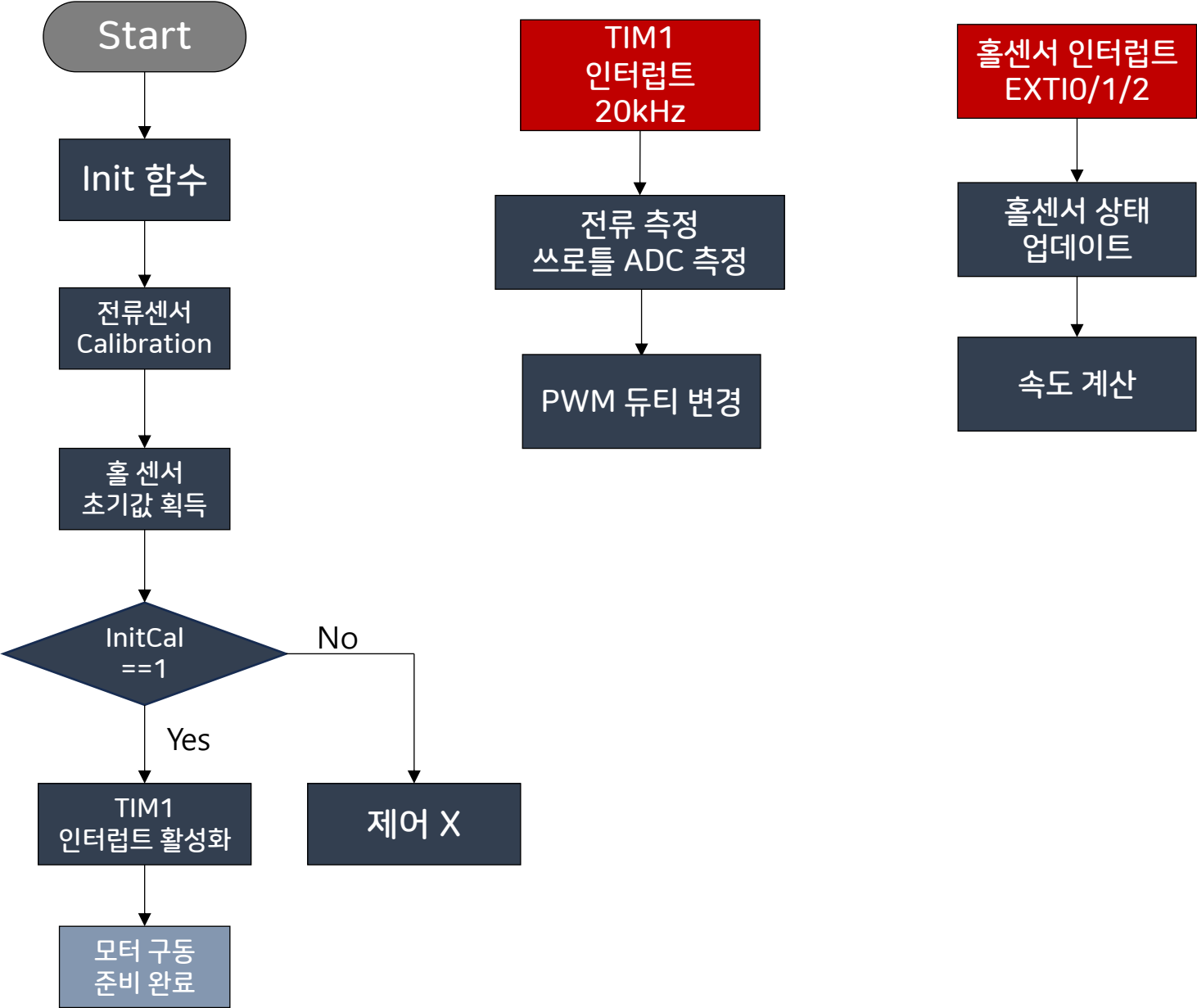
    USART3_SendString("Flt :");
    USART3_SendChar(FltFlg+48);
    // USART3_SendString("\n");
    Task_500msFlg=0;
}
```

- 블루투스 모듈(AT-09)
 - 문자열 송신 함수와 문자 송신 함수를 이용하여 블루투스 모듈에게 데이터 전송

```
void USART3_SendChar(char c)
{
    // TXE=1 (전송 데이터 레지스터 비어있음) 대기
    while (!(USART3->ISR & USART_ISR_TXE));
    USART3->TDR = (uint8_t)c;
}

// 문자열 송신
void USART3_SendString(const char *str)
{
    while (*str)
    {
        USART3_SendChar(*str++);
    }
}
```

BLDC 모터 제어 플로우 차트

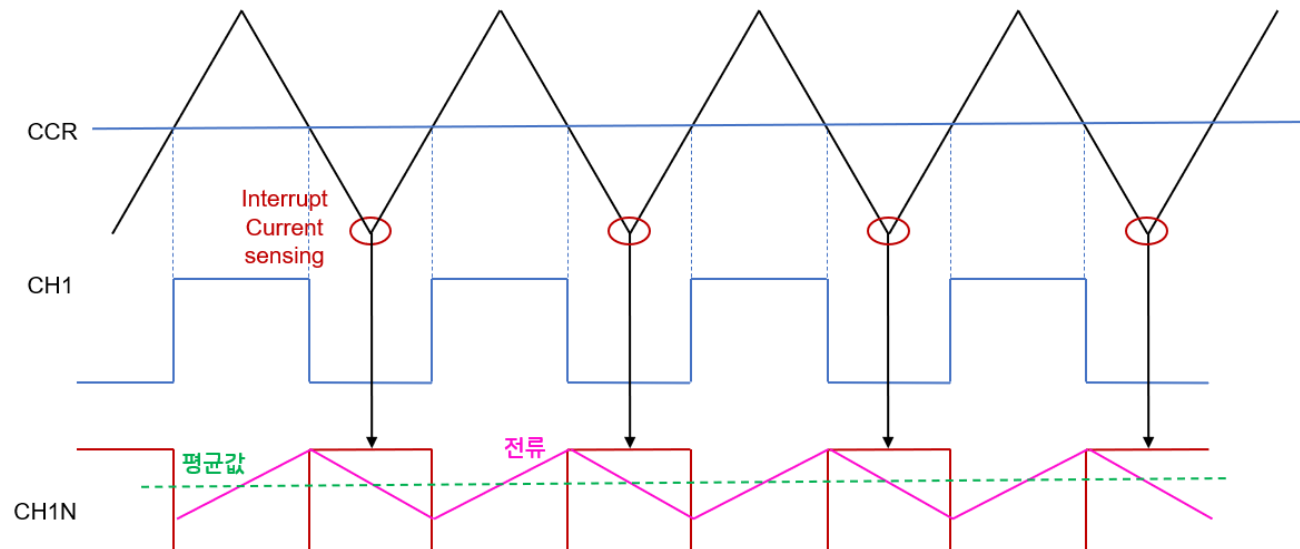


- BLDC 정/역 구동
 - 현재 코드에서는 dir변수 값으로 정방향, 역방향 회전을 변경할 수 있음
 - 인휠 모터의 경우 정방향, 역방향 기준을 잡기 애매해서 임의로 설정
 - 정방향과 역방향은 UVW상 스위칭 패턴을 반전 시키면 됨
 - ex) 홀센서가 010으로 들어온 경우 → 정방향 : 0,-1,+1 역방향 : 0,+1,-1

U상	0	1	1	0	-1	-1
V상	-1	-1	0	1	1	0
W상	+1	0	-1	-1	0	1
Ha	0	0	0	1	1	1
Hb	1	1	0	0	0	1
Hc	0	1	1	1	0	0
Sector	1(0~60°)	2(60~120°)	3(120~180°)	4(180~240°)	5(240~300°)	6(300~360°)
Hall Sum	2	3	1	5	4	6

- CNT_MAX(ARR 레지스터 값)에서 쓰로틀 adc신호로 얻는 VoltageRef를 빼 주는 이유
 - 만일 CNT_MAX에서 VoltageRef를 빼 주지 않으면 쓰로틀 adc 신호가 커질 수록 상단 스위치의 듀티가 작아지게 됨.
 - 하지만 우리가 원하는 건 쓰로틀 adc 신호에 비례하여 상단 스위치의 듀티를 상승 시키는 것이 목적이므로 CNT_MAX에서 VoltageRef를 빼서 원하는 동작을 수행시킴 → 쓰로틀 adc 신호가 커질수록 상단 스위치 듀티값이 커짐

```
void Set_Phases(int32_t phaseA, int32_t phaseB, int32_t phaseC)
{
    DutyA=CNT_MAX-VoltageRef;
    DutyB=CNT_MAX-VoltageRef;
    DutyC=CNT_MAX-VoltageRef;
}
```



- 고장 코드 정의
 - 현재 펌웨어는 아래와 같이 FltFlg가 정의 되어있음

FltFlg	내용
0	고장 x
1	과전류
2	MOSFET 과열
3	배터리 저전압
4	배터리 과전압

▪ BLDC 모터 구동 시 Mosfet 드레인-소스 파형 분석

